

p/classactivity/attempt?assignment_id=1284

SIMATS Online Programming Lab

PYTHON PROGRAMMING

9 Kampati Pavan
192524533RC

bala krishnan

Starts with A or D:

arun kumar

divya devi

Your Code

```
1 # Customer Data Analysis
2
3 names = [
4     "Arun Kumar ",
5     "ARUN KUMAR",
6     "arun kumar",
7     "Bala Krishnan",
8     "BALA KRISHNAN ",
9     "Charan",
10    " charan ",
11    "DIVYA DEVI",
12    "Divya Devi"
13 ]
14
15 # 1. Remove leading and trailing spaces
16 # 2. Convert all names to lowercase
17 normalized = [name.strip().lower() for name in names]
18
19 print("Normalized:")
20 print(normalized)
21
22 # 3. Identify duplicate customers
23 from collections import Counter
24
25 frequency = Counter(normalized)
26
27 duplicates = [
28     name for name in frequency
29     if frequency[name] > 1
30 ]
31
32 # 4. Count number of unique customers
```

Input

stdin input

Output

```
Normalized:
['arun kumar', 'arun kumar',
'arun kumar', 'bala krishnan',
'bala krishnan', 'charan',
'charan', 'divya devi', 'divya
devi']
```

Your Code

```
1 # Student Score Analysis
2
3 students = [
4     ("Arun", [78, 85, 92, 67]),
5     ("Bala", [45, 67, 55, 49]),
6     ("Charan", [90, 92, 88, 95]),
7     ("Divya", [65, 72, 70, 68]),
8     ("Esha", [88, 76, 91, 84])
9 ]
10
11 # 1. Calculate total and average marks
12 totals = {}
13 averages = {}
14
15 for name, marks in students:
16     total = sum(marks)
17     average = total / len(marks)
18     totals[name] = total
19     averages[name] = average
20
21 print("Student Totals:")
22 for name, total in totals.items():
23     print(f"{name} : {total}")
24
25 print()
26
27 print("Student Averages:")
28 for name, average in averages.items():
29     print(f"{name} : {average:.2f}")
30
31 # 2. Student with highest total
32 highest_total_student = max(totals, key=totals.get)
33
34 # 3. Student with lowest average
35 lowest_average_student = min(averages, key=averages.get)
36
37 print()
38 print(f"Highest Total: {highest_total_student}")
```

Input

stdin input

Output

```
Student Totals:
Arun : 322
Bala : 216
Charan : 365
Divya : 275
Esha : 338
```

```

12 products = [
13     ("Laptop", 5, 55000),
14     ("Mouse", 25, 700),
15     ("Keyboard", 8, 1200),
16     ("Monitor", 12, 15000),
17     ("Printer", 4, 12000),
18     ("Headset", 15, 2500)
19 ]
20
21 # 1. Calculate inventory value of every product
22 print("Inventory Values:")
23
24 inventory_values = {}
25
26 for product, quantity, price in products:
27     value = quantity * price
28     inventory_values[product] = value
29     print(f"{product}: {value}")
30
31 # 2. Calculate total inventory value
32 total_inventory_value = sum(inventory_values.values())
33
34 print()
35 print(f"Total Inventory Value: {total_inventory_value}")
36
37 # 3. Identify products whose quantity is less than 10
38 print()
39 print("Low Stock:")
40
41 for product, quantity, price in products:
42     if quantity < 10:
43         print(product)

```

Output

```

Inventory Values:
Laptop: 275000
Mouse: 17500
Keyboard: 9600
Monitor: 180000
Printer: 48000

```



```

1 text = ""
2 Python programming is easy to learn.
3 Python programming is powerful.
4 Learning Python requires practice.
5 Practice makes programming skills better.
6 """
7
8 # 1. Convert entire text to lowercase
9 text = text.lower()
10
11 # 2. Remove punctuation
12 punctuation = ".,:;\\"
13
14 for char in punctuation:
15     text = text.replace(char, "")
16
17 # 3. Convert text into a list of words
18 words = text.split()
19
20 # 4. Count total number of words
21 total_words = len(words)
22
23 # 5. Find number of unique words
24 unique_words = set(words)
25 number_of_unique_words = len(unique_words)
26
27 # 6. Frequency of "python"
28 python_frequency = words.count("python")
29
30 # 7. Frequency of "programming"
31 programming_frequency = words.count("programming")
32
33 # 8. Identify words appearing more than once

```

stdin input

Output

```

Total Words: 19
Unique Words: 13
Frequency of python: 3
Frequency of programming: 3

```

```

10 employees = [
11     ("E101", "Arun", ["Python", "SQL", "ML"]),
12     ("E102", "Bala", ["Java", "SQL", "HTML"]),
13     ("E103", "Charan", ["Python", "HTML", "SQL"]),
14     ("E104", "Divya", ["Python", "Java", "Cloud"]),
15     ("E105", "Esha", ["Python", "ML", "Cloud"])
16 ]
17
18 # 1. Find all employees who know Python
19 python_employees = [
20     name for emp_id, name, skills in employees
21     if "Python" in skills
22 ]
23
24 print("Python:")
25 print(", ".join(python_employees))
26
27 # 2. Find employees who know both Python and ML
28 python_ml = [
29     name for emp_id, name, skills in employees
30     if "Python" in skills and "ML" in skills
31 ]
32
33 print()
34 print("Python + ML:")
35 print(", ".join(python_ml))
36
37 # 3. Find employees who know SQL but not Java
38 sql_not_java = [
39     name for emp_id, name, skills in employees
40     if "SQL" in skills and "Java" not in skills
41 ]
42
43 print()
44 print("SQL but not Java:")
45 print(", ".join(sql_not_java))

```

stdin input

Output

```

Python:
Arun, Charan, Divya, Esha

Python + ML:
Arun, Esha

```

```

1 # Transaction Analysis - Tuple - List
2
3 transactions = [
4     ("T101", "Deposit", 5000),
5     ("T102", "Withdraw", 2000),
6     ("T103", "Deposit", 8000),
7     ("T104", "Withdraw", 1500),
8     ("T105", "Deposit", 3000),
9     ("T106", "Withdraw", 7000)
10 ]
11
12
13 # 1. Calculate total deposits
14 total_deposits = sum(
15     amount for tid, transaction_type, amount in transactions
16     if transaction_type == "Deposit"
17 )
18
19 # 2. Calculate total withdrawals
20 total_withdrawals = sum(
21     amount for tid, transaction_type, amount in transactions
22     if transaction_type == "Withdraw"
23 )
24
25 # 3. Calculate net balance
26 net_balance = total_deposits - total_withdrawals
27
28
29 # 4. Find the largest deposit
30 largest_deposit = max(
31     amount for tid, transaction_type, amount in transactions
32     if transaction_type == "Deposit"
33 )
34
35 # 5. Find the largest withdrawal
36 largest_withdrawal = max(
37     amount for tid, transaction_type, amount in transactions

```

stdin input

Output

Your Code

```
1 # Password Validation
2
3 passwords = [
4     "Python@123",
5     "python123",
6     "PYTHON@123",
7     "Py@45",
8     "Python123",
9     "Java#4567",
10    "Admin@2026"
11 ]
12
13
14 # Password validation function
15 def validate_password(password):
16
17     conditions = {
18         "uppercase": any(char.isupper() for char in password),
19         "lowercase": any(char.islower() for char in password),
20         "digit": any(char.isdigit() for char in password),
21         "special": any(not char.isalnum() for char in password),
22         "length": len(password) >= 8
23     }
24
25     return conditions
26
27
28 valid_passwords = []
29 invalid_passwords = []
30
31
32 # Check every password
33 for password in passwords:
34
35     conditions = validate_password(password)
36
37     failed = []
38
```

Input

stdin input

Output

```
Valid:
Python@123
Java#4567
Admin@2026
Invalid:
```

Why does 'bala' appear only once?

7. What is the final output?

8. Modify the logic to find unique employees and their skills.

9. Analyze what happens if record[0] is changed instead.

10. Explain why modifying record[0] directly is not allowed.

Expected Output:

['arun', 'bala', 'charan']

Your Code

```
1 records = [  
2     ("Arun", "Python"),  
3     ("ARUN", "Python"),  
4     ("Bala", "Java"),  
5     ("bala", "Java"),  
6     ("Charan", "Python")  
7 ]  
8  
9 names = []  
10  
11 for record in records:  
12     name = record[0].strip().lower()  
13  
14     if name not in names:  
15         names.append(name)  
16  
17 print(names)
```

Input

stdin input

Output

```
['arun', 'bala', 'charan']
```

Run

Save

10/10 answered

```
1 # 1. Convert sentence into a list of words
2
3 text = "data science uses data analysis and data visualization"
4
5 # 1. Convert sentence into a list of words
6 words = text.split()
7
8 # 2. Count total words
9 total_words = len(words)
10
11 # 3. Find unique words
12 unique_words = list(dict.fromkeys(words))
13
14 # 4. Count frequency of every word
15 frequency = {}
16
17 for word in words:
18     if word in frequency:
19         frequency[word] += 1
20     else:
21         frequency[word] = 1
22
23 # 5. Find the most frequently occurring word
24 most_frequent = max(frequency, key=frequency.get)
25
26 # 6. Create tuples of the form (word, frequency)
27 word_tuples = []
28
29 for word, count in frequency.items():
30     word_tuples.append((word, count))
31
32 # 7. Sort tuples based on frequency
33 sorted_tuples = sorted(word_tuples, key=lambda x: x[1], reverse=True)
```

stdin input

Output

```
Total Words: 8
Unique Words:
['data', 'science', 'uses',
'analysis', 'and',
'visualization']
```

your code

```
students = [
    ("A001 ", "ARUN KUMAR", [78, 85, 92]),
    ("A002", "Bala", [45, 67, 52]),
    ("a003 ", "arun kumar", [88, 91, 95]),
    ("A004", "CHARAN", [72, 76, 81]),
    ("A005", "Divya", [55, 48, 62])
]

# -----
# 1. Normalize student IDs and names
# -----

normalized_students = []

for student_id, name, marks in students:
    student_id = student_id.strip().upper()
    name = name.strip().lower()
    normalized_students.append(
        (student_id, name, marks)
    )

print("Normalized Records:")

for student_id, name, marks in normalized_students:
    print(student_id, name, marks)

# -----
# 2. Calculate total and average marks
# -----

totals = {}
averages = {}
```

input

stdin input

Output

```
Normalized Records:
A001 arun kumar [78, 85, 92]
A002 Bala [45, 67, 52]
A003 arun kumar [88, 91, 95]
A004 charan [72, 76, 81]
A005 divya [55, 48, 62]
```